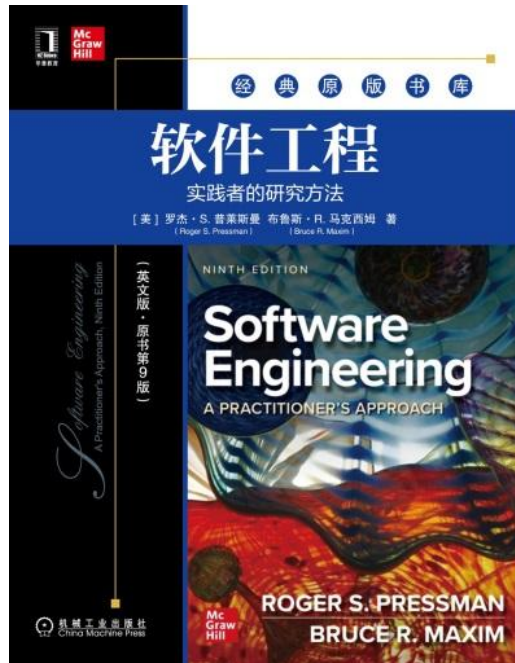


Advanced Software Engineering

Sichuan University
luoyining@scu.edu.cn

Text Book

- Software Engineering: A Practitioner's Approach, ninth edition, Roger S. Pressman, China Machine Press



CHAPTER 1 Software and Software Engineering 1

PART ONE THE SOFTWARE PROCESS 19

CHAPTER 2 Process Models 20
CHAPTER 3 Agility and Process 37
CHAPTER 4 Recommended Process Model 54
CHAPTER 5 Human Aspects of Software Engineering 74

PART TWO MODELING 83

CHAPTER 6 Principles That Guide Practice 84
CHAPTER 7 Understanding Requirements 102
CHAPTER 8 Requirements Modeling—A Recommended Approach 126
CHAPTER 9 Design Concepts 156
CHAPTER 10 Architectural Design—A Recommended Approach 181
CHAPTER 11 Component-Level Design 206
CHAPTER 12 User Experience Design 233
CHAPTER 13 Design for Mobility 264
CHAPTER 14 Pattern-Based Design 289

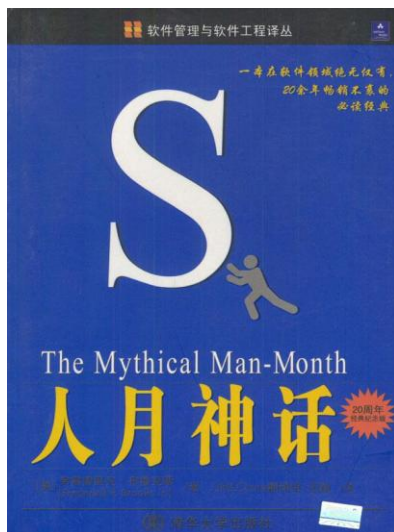
PART THREE QUALITY AND SECURITY 309

CHAPTER 19 Software Testing—Component Level 372
CHAPTER 20 Software Testing—Integration Level 395
CHAPTER 21 Software Testing—Specialized Testing for Mobility 412

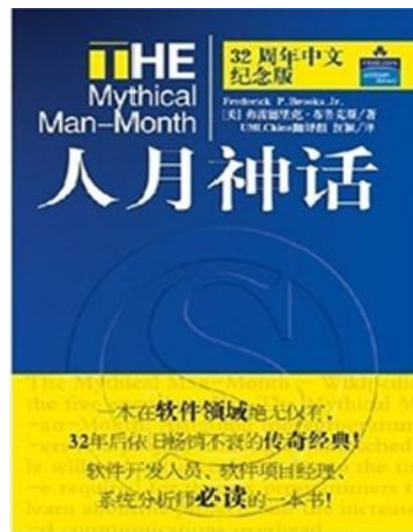
APPENDIX 1 An Introduction to UML 611

APPENDIX 2 Data Science for Software Engineers 629

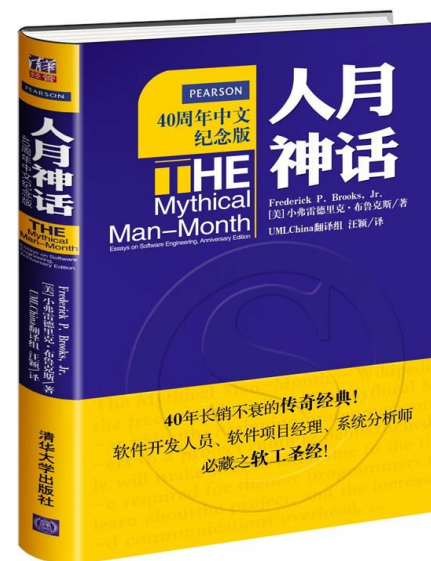
Reference Books



20周年纪念版



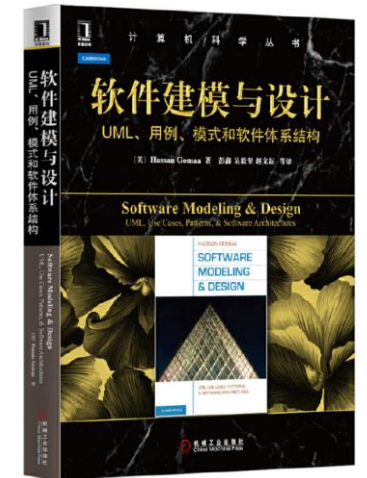
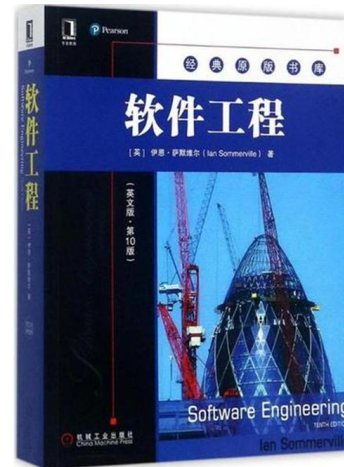
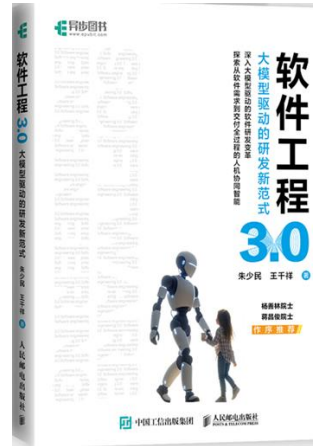
32周年纪念版



40周年纪念版

- 技术并非《人月神话》的着眼点，它更关注的是软件的创造过程、需求的变化无常和管理的永恒困境；
- 《人月神话》的中心思想已经超越了具体的时代和技术。

Reference Books



Learning by doing (做中学)

- 教育是令人羡慕的东西，但是要记住：凡是值得知道的，没有一个是能够教会的。

——奥斯卡·王尔德 (Oscar Wilder)

- 从错误中学习：不断改进，先有量变，再有质变
- 和别人合作：1:1 1:many
- 项目安排
 - I: Individual project
 - P: Pair Project
 - T: Team Project



Grading

- 总成绩 = 期末考试 50% (强制达标线: 50分)
 - + 平时测验 20%
 - + 平时成绩 30%

Chapter 1

Software and Software Engineering

Key Points

- Software
- Software's dual roles
- Software vs. hardware
- Software engineering
- Software engineering layers
- Five framework activities
- Umbrella activities
- Software myths

The Nature of Software

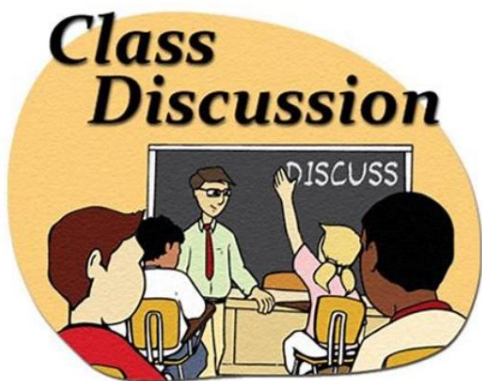
- 软件产业已成为经济的主导因素。

早期的独立程序员已被专业的软件开发团队代替。

但是，软件开发人员依然面临同样的问题：

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?

课堂讨论：软件 vs 程序

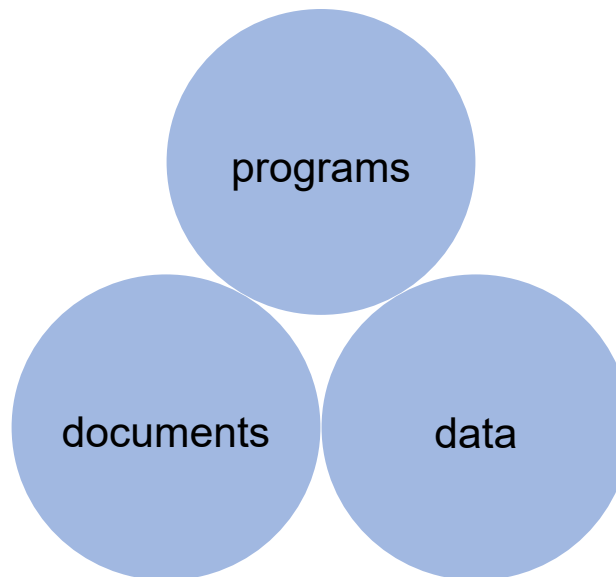


如何理解 “软件” 与 “程序” 之间的区别？

Definition

Software is:

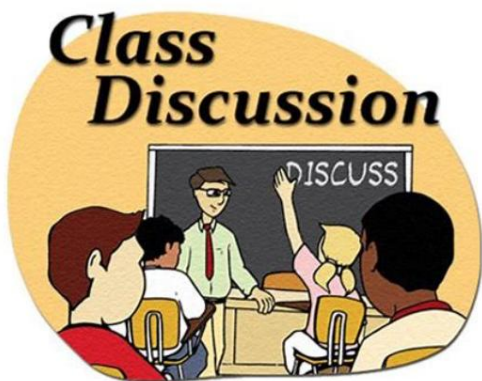
- (1) **instructions** (computer programs) that when executed provide desired features, function, and performance; 通过执行程序来满足预期的特征、功能和性能需求。
- (2) **data structures** that enable the programs to adequately manipulate information;
- (3) **descriptive information** that describes the operation and use of the programs.



Software's Dual Roles

- Software is a product.
 - Delivers the computing potential
 - Produces, manages, acquires, modifies, displays, or transmits information
- Software is a vehicle for delivering a product. 软件是产品生产的载体
 - The control of the computer (e.g., an operating system)
 - The communication of information (e.g., networking software)
 - The creation and control of other programs (e.g., software tools & environment)

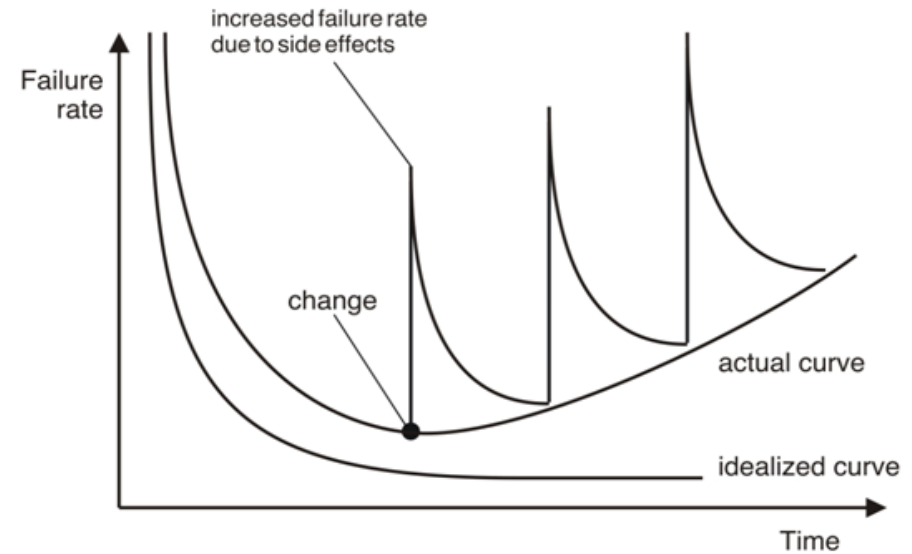
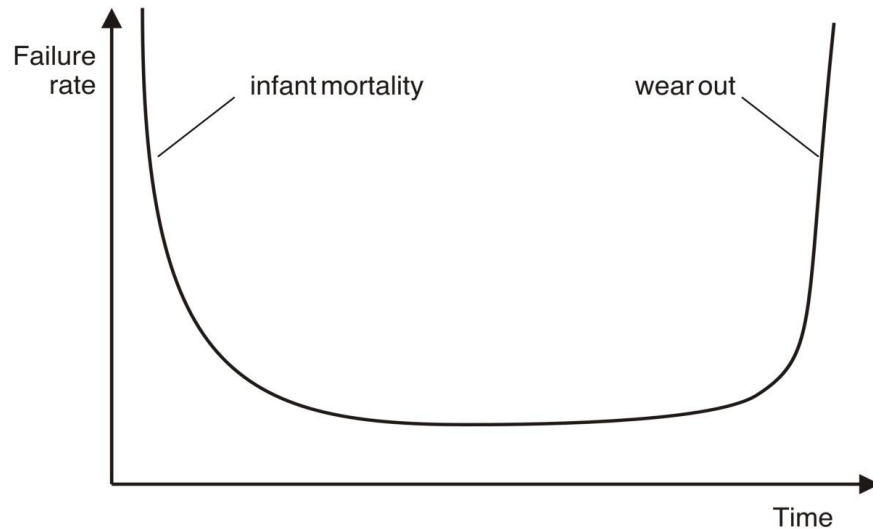
课堂讨论：软件 vs 硬件



如何理解 “软件” 与 “硬件” 之间的区别？

Software vs. Hardware

- Software has one fundamental characteristic that make it considerably different from hardware: **Software doesn't “wear out”**.



Wear Out vs. Deterioration (退化)

Software Applications

- System software
- Application software
- Engineering/scientific software
- Embedded software
- Product-line software
- Web/Mobile applications
- AI software (robotics, neural nets, game playing)

Legacy Software

- Longevity
- Business criticality
- Poor quality
 - Inextensible designs
 - Convoluted code
 - Poor or nonexistent documentation
 - Test case and results that were never archived
 - A poorly managed change history

Do nothing.

Evolution

Why must software change ?

- As time passes, legacy systems often evolve for one or more of the following reasons:
 - Software must be **adapted** to meet the needs of new computing environments or technology.
 - Software must be **enhanced** to implement new business requirements.
 - Software must be **extended** to make it interoperable with other more modern systems or databases.
 - Software must be **re-architected** to make it viable within a network environment.
- **Question**
 - Explain what is wrong with the notion that computer software does not need to evolve over time.

Year 2000 Problem

Y2K Bug / Millennium Bug

A problem for both digital (computer-related) and non-digital documentation and data storage situations which resulted from the practice of abbreviating a four-digit year to two digits.



Software Crisis

- Software crisis is a term used in the early days of computing science for the difficulty of writing useful and efficient computer programs in the required time.
- The software crisis was due to the rapid increases in computer power and the complexity of the problems that could be tackled.
- With the increase in the complexity of the software, many software problems arose because existing methods were neither sufficient nor up to the mark.

https://en.wikipedia.org/wiki/Software_crisis

Typical Case

- IBM公司在1963年至1966年开发的IBM360机的操作系统。
 - 这一项目花了5000人一年的工作量，最多时有1000人投入开发工作，写出了近100万行源程序。
 - 据统计，这个操作系统每次发行的新版本都是从前一版本中找出1000个程序错误而修正的结果。
 - 这个项目的负责人F. P. Brooks事后总结了他在组织开发过程中的沉痛教训时说：“.....程序设计工作正像这样一个泥潭，.....一批批程序员被迫在泥潭中拼命挣扎，.....谁也没有料到问题竟会陷入这样的困境.....”。

The Tar Pit (焦油坑)



拉布雷阿的焦油坑壁画

在史前史中，没有别的场景比巨兽在焦油坑中垂死挣扎的场面更令人震撼。上帝见证着恐龙、猛犸象、剑齿虎在焦油中挣扎。它们挣扎得越是猛烈，焦油纠缠得越紧，没有任何猛兽足够强壮或具有足够的技巧，能够挣脱束缚，它们最后都沉到了坑底。

过去几十年的大型系统**开发就犹如这样一个焦油坑**，很多大型和强壮的动物在其中剧烈地挣扎。他们中大多数开发出了可运行的系统——不过，其中只有非常少数的项目满足了目标、时间进度和预算的要求。各种团队，大型的和小型的，庞杂的和精干的，一个接一个淹没在了焦油坑中。表面上看起来好像没有任何一个单独的问题会导致困难，每个都能被解决，但是当它们相互纠缠和累积在一起的时候，团队的行动就会变得越来越慢。

—— F. Brooks, 《人月神话》，清华大学出版社



Frederick Phillips Brooks. Jr.

- 美国工程院院士
- IBM360系统之父，曾担任360系统的项目经理，及该项目设计阶段的经理。凭借在此项目中的杰出贡献，在1985年获得美国国家技术奖。
- 1999年获图灵奖。

Software Crisis

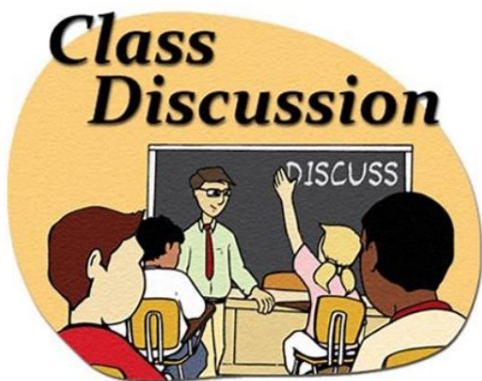
- 为了克服软件危机，1968年10月，在北大西洋公约组织（NATO）召开的计算机科学会议上，Fritz Bauer首次提出“软件工程”的概念，试图将工程化方法应用于软件开发。

—— 郑人杰等，《软件工程概论》，机械工业出版社

水利工程 建筑工程 机械工程

气象工程 生物工程 **软件工程**

课堂讨论：工程化软件开发



如何理解“工程化软件开发”与到目前为止大家所熟悉的“编程”之间的区别？



Definition [IEE17]

- The IEEE definition for software engineering:

The application of a **systematic**, **disciplined**, **quantifiable** approach to the **development**, **operation**, and **maintenance** of software; that is, the application of engineering to software.

[IEE17] ISO/IEC/IEEE 24765:2017(E), *ISO/IEC/IEEE International Standard: Systems and Software Engineering—Vocabulary*, available at <https://standards.ieee.org/findstds/standard/24765-2017.html>.

Definition [IEE93a]

- The IEEE definition for software engineering:
 - (1) The application of a **systematic**, **disciplined**, **quantifiable** approach to the **development**, **operation**, and **maintenance** of software; that is, the application of engineering to software.
 - (2) The study of approaches as in (1).

[IEE93a] *IEEE Standards Collection: Software Engineering*, IEEE Standard 610.12-1990, IEEE, 1993.

如何理解软件工程

- 软件工程可以理解为软件开发的工程化或工程化的软件开发
- 基本追求：质量、成本、效率
- “工程”的基本内涵
 - **过程标准化**：通常包含一系列明确定义的过程，包括相关的活动、使用的工具、参与的角色以及具体工作的指南等，相关过程经常可以通过标准和规范等手段进行明确总结和描述
 - **理论和实践支撑**：所涉及的原则和方法有相应的理论支撑或者体现了长期实践经验的总结
 - **质量有保障**：项目实施过程稳定、可控，所完成项目或所交付产品的质量在很大程度上能够得到保障，成功经验很大程度上可总结、可学习、可重复
 - **追求实用**：遵循实用性原则，强调“足够好”（good enough）而非“完美”（perfect），注重成本与效益等方面的权衡



A Layered Technology

- Software engineering is a layered technology.



Quality Focus is a bedrock to support software engineering.
支持软件工程的根基在于质量关注点。

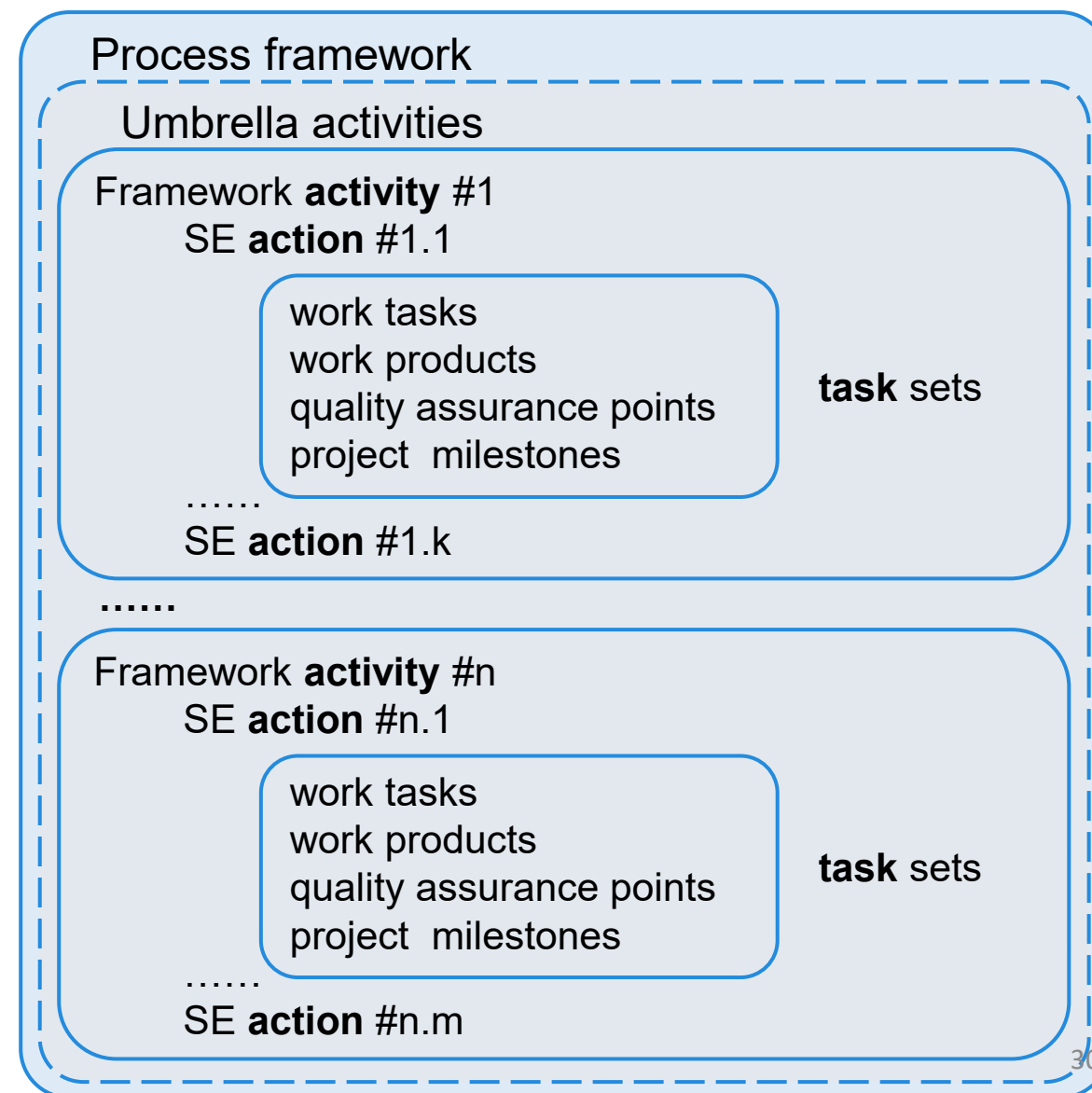
Software Process

- A process is a collection of **activities**, **actions**, and **tasks** that are performed when some work product is to be created.
- Each of these activities, actions, and tasks reside within a **framework** or **model** that defines their relationship with the process and with one another.

Process Framework

任何一个软件过程都可以用过程框架来描述。

- 活动主要实现宽泛的目标，如与利益相关者进行沟通，与应用领域、项目大小等没有直接关系。
- 动作，如体系结构设计，包含一系列具体任务。
- 任务关注小而明确的目标，能够产生实际产品，如构建一个单元测试。



A Generic Process Framework

Generic Framework Activities

框架活动广泛应用于所有软件开发项目，无论项目的规模和复杂性如何。

- Communication
- Planning
- Modeling
- Construction
- Deployment

Umbrella Activities

普适性活动贯穿软件项目始终，帮助团队管理和控制项目进度、质量、变更和风险。

- Software project tracking and control
- Risk management
- Software quality assurance
- Technical reviews
- Measurement
- Software configuration management
- Reusability management
- Work product preparation and production

Process Adaptation

- The framework activities will always be applied on every project ... BUT the software engineering process should be **agile** and **adaptable**
 - to the problem, to the project, to the team, and to the organizational culture.
- A process adopted for one project might be significantly **different** than a process for another project.

The Essence of Practice

- 《How to solve it》
 - 1. Understand the problem (communication and analysis)
 - 2. Plan a solution (modeling and software design)
 - 3. Carry out the plan (code generation)
 - 4. Examine the result for accuracy (testing and quality assurance)

Hooker's General Principles

- David Hooker[Hoo96] 提出了7个关注软件工程整体实践的原则:
 1. The Reason It All Exists
 2. KISS (Keep It Simple, Stupid!)
 3. Maintain the Vision
 4. What You Produce, Others Will Consume
 5. Be Open to the Future
 6. Plan Ahead for Reuse
 7. Think!

Software Myths

- 软件神话是指关于软件及其开发过程被人盲目相信的一些说法。

Manager	Myth	If we get behind schedule, we can add more programmers and catch up.
	Reality	• Software development is not a mechanistic process like manufacturing. • “Adding people to a late software project makes it later.” [Bro95] • People can be added only in a planned and well-coordinated manner.
Customer	Myth	Project requirements continually change, but change can be easily accommodated because software is flexible.
	Reality	• The impact of change varies with the time at which it is introduced. As time passes, cost impact grows rapidly. • Change can cause upheaval that requires additional resources and major design modification.
Practitioner	Myth	Until I get the program running, I have no way of assessing its quality.
	Reality	• The formal technical review is one of most effective quality assurance mechanisms. • Software reviews are a “quality filter” that have been found to be more effective than testing for finding certain classes of software errors.

How It all Starts

- Every software project is precipitated by some business need —
 - the need to correct a defect in an existing application;
 - the need to adapt a 'legacy system' to a changing business environment;
 - the need to extend the functions and features of an existing application, or
 - the need to create a new product, service, or system.

SafeHome

How a Project Starts

The scene:

The players:

The conversation:



Summary

- Software
- Software's dual roles
- Software vs. hardware
- Software engineering
- Software engineering layers
- Five framework activities
- Umbrella activities
- Software myths

Problems and Points to Ponder

- 1.1. Provide at least five additional examples of how the law of unintended consequences applies to computer software.
- 1.3. Develop your own answers to the five questions asked at the beginning of Section 1.1. Discuss them with your fellow students.
- 1.7. Describe a process framework in your own words. When we say that framework activities are applicable to all projects, does this mean that the same work tasks are applied for all projects, regardless of size and complexity? Explain.